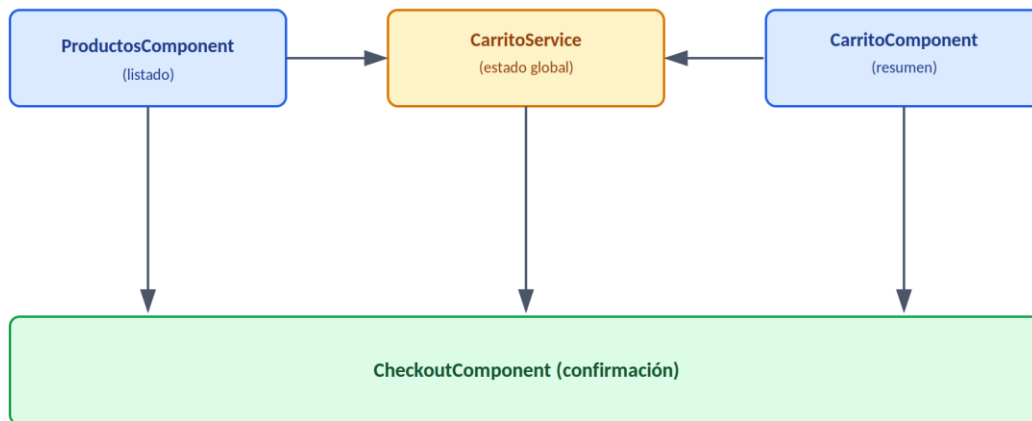


Documentación del Carrito de Ventas con Observables y Pipes

1. Diseño del Flujo del Carrito

El flujo del carrito de ventas se compone de tres componentes principales que interactúan a través de un servicio central:



Etapas del flujo:

1. Selección: El usuario navega al listado de productos y agrega productos al carrito mediante el botón “Agregar”.
2. Revisión: El usuario accede al carrito donde puede modificar cantidades, eliminar productos o proceder al pago.
3. Confirmación: En el checkout se muestra el resumen final y se confirma la orden, que se persiste en la base de datos PostgreSQL.

2. Rol del Servicio (CarritoService)

El CarritoService actúa como fuente única de verdad del estado del carrito, sus responsabilidades son:

- **Centralizar el estado:** Mantiene el array de productos del carrito en un BehaviorSubject.
- **Exponer actualizaciones:** Provee un observable público (carrito\$) al que los componentes se suscriben.
- **Gestionar operaciones:** Implementa los métodos agregarProducto(), actualizarCantidad(), eliminarProducto() y limpiarCarrito().
- **Comunicar con el backend:** Envía la orden confirmada al endpoint POST /api/ordenes.
- **Calcular totales:** Ofrece métodos auxiliares como getTotal() y getCantidadTotal().

Al ser proporcionado con providedIn: "root", Angular garantiza una única instancia compartida en toda la aplicación, lo que asegura consistencia de datos entre componentes.

3. Uso de Observables

Se utiliza un BehaviorSubject<ProductoCarrito[]> como mecanismo reactivo:

```
private carritoSubject = new BehaviorSubject<ProductoCarrito[]>([]);
carrito$ = this.carritoSubject.asObservable();
```

Ventajas del enfoque reactivo:

- **Propagación automática:** Cada vez que se llama a carritoSubject.next(), todos los suscriptores reciben la actualización sin necesidad de recargar la página.
- **Valor inicial:** BehaviorSubject emite inmediatamente el último valor a nuevos suscriptores, evitando estados vacíos al navegar entre componentes.
- **Desacoplamiento:** Los componentes no se comunican directamente entre sí, sino a través del servicio, lo que facilita el mantenimiento y las pruebas.

Gestión de suscripciones:

Cada componente suscriptor almacena la suscripción y la libera en ngOnDestroy() para evitar fugas de memoria:

```
ngOnDestroy(): void {
  if (this.subscription) {
    this.subscription.unsubscribe();
  }
}
```

4. Pipes Implementados

Se implementaron tres pipes personalizados para los cálculos del carrito:

4.1. SubtotalPipe

Calcula el subtotal de un producto individual multiplicando precio por cantidad.

```
@Pipe({ name: "subtotal", standalone: true })
export class SubtotalPipe implements PipeTransform {
  transform(precio: number, cantidad: number): number {
    if (!precio || !cantidad) return 0;
    return precio * cantidad;
  }
}
```

Uso en plantilla:

```
Q{{ item.precio | subtotal:item.cantidad | number:"1.2-2" }}
```

4.2. TotalCarritoPipe

Calcula el total general sumando los subtotales de todos los productos del carrito.

```
@Pipe({ name: "totalCarrito", standalone: true })
export class TotalCarritoPipe implements PipeTransform {
  transform(items: ProductoCarrito[]): number {
    if (!items || items.length === 0) return 0;
    return items.reduce((total, item) => total + (item.precio * item.cantidad), 0);
  }
}
```

Uso en plantilla:

```
Q{{ items | totalCarrito | number:"1.2-2" }}
```

4.3. CantidadTotalPipe

Calcula la cantidad total de unidades en el carrito sumando las cantidades de cada producto.

```
@Pipe({ name: "cantidadTotal", standalone: true })
export class CantidadTotalPipe implements PipeTransform {
  transform(items: ProductoCarrito[]): number {
    if (!items || items.length === 0) return 0;
    return items.reduce((total, item) => total + item.cantidad, 0);
  }
}
```

Uso en plantilla:

```
{{ items | cantidadTotal }}
```

5. Contribución a la Claridad de los Cálculos

Los pipes contribuyen a la claridad de la interfaz de las siguientes formas:

4. Separación de responsabilidades: La lógica de cálculo vive en los pipes, no en los componentes, manteniendo los componentes enfocados en la presentación.
5. Reutilización: Los mismos pipes se usan en múltiples componentes (carrito y checkout) sin duplicar código.
6. Legibilidad en plantillas: Las expresiones como `item.precio | subtotal:item.cantidad` son autoexplicativas y fáciles de leer.
7. Pureza: Los pipes son puros por defecto, lo que significa que Angular solo los re-ejecuta cuando cambian los inputs, optimizando el rendimiento.
8. Testabilidad: Cada pipe puede probarse de forma aislada con pruebas unitarias simples.